TRAMIA

Validator Node

Technical Implementation Guide

Document Version	1.0
Classification	Technical - Internal
Blockchain Platform	Hyperledger Besu (Permissioned)
Consensus Mechanism	QBFT (Quorum Byzantine Fault Tolerant)
Target Audience	Technical Teams, DevOps, Infrastructure
Last Updated	January 2026

Prepared by Riverbank Solutions Limited

Table of Contents

Table of Contents....2
1. Executive Summary....4
1.1 What is a Validator?....4
1.2 Validator Responsibilities....4
1.3 Why Run a Validator?....4
2. Network Architecture....5
2.1 Tramia Blockchain Overview....5
2.2 Network Topology....5
2.3 Validator Node Types....6
3. Consensus Mechanism (QBFT)....7
3.1 What is QBFT?....7
3.1.1 Key Properties....7
3.2 Consensus Requirements....7
3.3 Block Production Process....7
4. Validator Node Setup....9
4.1 Hardware Requirements....9
4.2 Network Requirements....9
4.3 Software Stack....9
4.4 Configuration Files....10
4.4.1 Genesis File (genesis.json)....10
4.4.2 Node Configuration (config.toml)....10
5. Key Management & Security....12
5.1 Validator Key Overview....12
5.2 Key Storage Options....12
5.3 Key Generation Ceremony....12
5.4 Security Best Practices....13
6. Deployment Options....14
6.1 Deployment Comparison....14
6.2 Cloud Deployment (Recommended for Starting)....14
6.3 Docker Compose (Development/Testing)....15
7. Operations & Maintenance....17
7.1 Operational Model....17
7.2 Staffing Requirements....17
7.3 Monitoring & Alerting....18
7.3.1 Key Metrics to Monitor....18

7.3.2 Alert Configuration 18
8. Network Governance 20
8.1 Governance Model 20
8.2 Governance Actions 20
8.3 Voting Process 20
9. Troubleshooting Guide 22
9.1 Common Issues and Solutions 22
9.1.1 Node Won't Start 22
9.1.2 No Peer Connections 22
9.1.3 Not Producing Blocks 23
9.2 Emergency Procedures 23
10. Getting Started Checklist 25
10.1 Contact Information 25
Appendix A: Glossary 26

1. Executive Summary

This document provides a comprehensive technical guide for organizations participating in the Tramia blockchain network as validator nodes. Validators are the backbone of the network, responsible for transaction validation, block production, and maintaining consensus.

1.1 What is a Validator?

A validator is a network participant that runs specialized node software to secure and operate the Tramia blockchain. Unlike public blockchains where anyone can become a validator, Tramia uses a permissioned model where validators are pre-approved organizations that have a stake in the network's integrity.

> KEY POINT: Validators operate autonomously 24/7 with minimal human intervention. They do NOT require dedicated operators watching them constantly.

1.2 Validator Responsibilities

Responsibility	Description
Block Production	Take turns proposing new blocks containing validated transactions
Consensus Voting	Vote on proposed blocks using QBFT consensus algorithm
Transaction Validation	Verify signatures, check compliance rules, validate smart contract execution
State Maintenance	Maintain authoritative state of all wallets, balances, and asset ownership
Network Security	Prevent double-spending, enforce tokenization rules, maintain audit trail
Data Availability	Store and serve blockchain data to other network participants

1.3 Why Run a Validator?

- Network Stake: Direct participation in network governance and operations
- Transparency: Full visibility into all transactions on the network
- Independence: No reliance on third parties for transaction verification
- Regulatory Compliance: Ability to meet regulatory requirements for data sovereignty
- Business Continuity: Ensures network remains operational even if other validators fail

2. Network Architecture

2.1 Tramia Blockchain Overview

Tramia is built on Hyperledger Besu, an enterprise-grade Ethereum client designed for permissioned networks. It provides EVM compatibility while offering the control and privacy required for regulated financial services.

Parameter	Value
Platform	Hyperledger Besu 24.x
Network Type	Permissioned (private)
Consensus	QBFT (Quorum Byzantine Fault Tolerant)
Block Time	2 seconds
Finality	Immediate (no probabilistic confirmation)
Smart Contracts	Solidity (EVM compatible)
Token Standards	ERC-20, ERC-721, ERC-3643 (security tokens)
Chain ID	2026 (Tramia mainnet)

2.2 Network Topology

The Tramia network consists of validator nodes operated by approved organizations. Each validator maintains a full copy of the blockchain and participates in consensus.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        TRAMIA VALIDATOR NETWORK                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│        ┌──────────────────┐                ┌──────────────────┐         │
│        │   VALIDATOR 1    │◄──────────────►│   VALIDATOR 2    │         │
│        │   (Riverbank)    │                │   (Partner Bank) │         │
│        │   Primary Node   │                │   Partner Node   │         │
│        └──────────┬───────┘                └──────────┬───────┘         │
│                   │                                   │                 │
│                   │       P2P Mesh Network            │                 │
│                   │       QBFT Consensus              │                 │
│                   │       2-second blocks             │                 │
│                   │                                   │                 │
│        ┌──────────┴───────┐                ┌──────────┴───────┐         │
│        │   VALIDATOR 3    │◄──────────────►│   VALIDATOR 4    │         │
│        │   (DR Site)      │                │   (Regulatory)   │         │
│        │   Backup Node    │                │   Observer Node  │         │
│        └──────────────────┘                └──────────────────┘         │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

```
│   All validators are equal participants in consensus                    │
│   Each validator stores full blockchain state                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

2.3 Validator Node Types

Node Type	Role	Operator
Primary	Main validator for organization	Riverbank Solutions
Partner	Bank/financial institution validator	Partner Bank
Backup/DR	Disaster recovery validator	Riverbank (secondary site)
Regulatory	Read-only observer for compliance	Regulator (CMA/CBK)

3. Consensus Mechanism (QBFT)

3.1 What is QBFT?

QBFT (Quorum Byzantine Fault Tolerant) is a consensus algorithm designed for permissioned blockchain networks. It provides immediate transaction finality and can tolerate up to one-third of validators being faulty or malicious.

3.1.1 Key Properties

- Immediate Finality: Once a block is committed, it cannot be reversed
- Byzantine Fault Tolerance: Network continues operating even with malicious validators
- Deterministic: Same transactions always produce same results
- Leader-Based: Validators take turns proposing blocks in round-robin fashion

3.2 Consensus Requirements

QBFT requires a minimum number of validators to agree before a block is finalized:

FORMULA: Minimum validators to reach consensus = $\lfloor (n-1)/3 \rfloor + 1$, where n = total validators

Total Validators	Required for Consensus	Fault Tolerance	Recommended?
3	2	0	No - no fault tolerance
4	**3**	**1**	**Yes - minimum viable**
5	4	1	Yes
7	5	2	Yes - production

3.3 Block Production Process

```
QBFT CONSENSUS FLOW (Each 2-second block)
═══════════════════════════════════════════════════════════════════════════


Round 1: PROPOSE
┌──────────────────────────────────────────────────────────────────────────┐
│  Designated leader collects pending transactions                         │
│  Leader creates block proposal                                           │
│  Leader broadcasts PREPREPARE message to all validators                  │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
```

```
Round 2: PREPARE
┌──────────────────────────────────────────────────────────────────────────┐
│  Each validator validates the proposed block                             │
│  If valid, validator broadcasts PREPARE message                          │
│  Wait for 2f+1 PREPARE messages (where f = max faulty validators)       │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
Round 3: COMMIT
┌──────────────────────────────────────────────────────────────────────────┐
│  Once enough PREPAREs received, validator broadcasts COMMIT              │
│  Wait for 2f+1 COMMIT messages                                           │
│  Block is finalized and added to chain                                   │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
Round 4: FINALITY
┌──────────────────────────────────────────────────────────────────────────┐
│  Block is FINAL - cannot be reversed                                     │
│  All validators update their state                                       │
│  Next validator becomes leader for next block                            │
└──────────────────────────────────────────────────────────────────────────┘
```

4. Validator Node Setup

4.1 Hardware Requirements

Component	Minimum	Recommended
CPU	4 cores (x86_64)	8+ cores
Memory	8 GB RAM	16-32 GB RAM
Storage	250 GB SSD	500 GB+ NVMe SSD
Network	100 Mbps	1 Gbps dedicated
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04/24.04 LTS

4.2 Network Requirements

Port	Protocol	Purpose
30303	TCP/UDP	P2P communication between validators
8545	TCP	JSON-RPC API (internal services only)
8546	TCP	WebSocket API (internal services only)
9545	TCP	Metrics endpoint (Prometheus)

SECURITY: Ports 8545, 8546, and 9545 should NEVER be exposed to the public internet. Only port 30303 should be accessible from other validators.

4.3 Software Stack

```
# Required Software
────────────────────────────────────────────────────────────────────────────
Component                Version         Purpose
────────────────────────────────────────────────────────────────────────────
Hyperledger Besu         24.1.0+         Blockchain node software
Docker                   24.0+           Container runtime
Docker Compose           2.20+           Container orchestration
  OR
Kubernetes               1.28+           Production orchestration

# Optional (Recommended)
────────────────────────────────────────────────────────────────────────────
Prometheus               2.45+           Metrics collection
Grafana                  10.0+           Metrics visualization
AWS CloudHSM / Azure KV  -               Hardware security module
```

```
Terraform               1.5+            Infrastructure as code
```

4.4 Configuration Files

4.4.1 Genesis File (genesis.json)

The genesis file defines the initial state of the blockchain and is identical across all validators:

```
{
  "config": {
    "chainId": 2026,
    "berlinBlock": 0,
    "londonBlock": 0,
    "qbft": {
      "blockperiodseconds": 2,
      "epochlength": 30000,
      "requesttimeoutseconds": 4
    }
  },
  "nonce": "0x0",
  "timestamp": "0x0",
  "extraData": "0xf87aa00000...VALIDATOR_ADDRESSES_RLP_ENCODED...",
  "gasLimit": "0x1fffffffffffff",
  "difficulty": "0x1",
  "mixHash": "0x63746963616c2062797a616e...",
  "coinbase": "0x0000000000000000000000000000000000000000",
  "alloc": {
    "0xTRAMIA_TREASURY_ADDRESS": {
      "balance": "0x200000000000000000000000000000000000000000"
    },
    "0xFEE_COLLECTOR_ADDRESS": {
      "balance": "0x0"
    }
  }
}
```

4.4.2 Node Configuration (config.toml)

```
# Tramia Validator Node Configuration
# ═══════════════════════════════════════════════════════════════════

# ───────────────────────────────────────────────────────────────────
# NETWORK SETTINGS
# ───────────────────────────────────────────────────────────────────
data-path="/data/besu"
genesis-file="/config/genesis.json"
network-id=2026
```

```
# P2P Configuration
p2p-enabled=true
p2p-host="0.0.0.0"
p2p-port=30303
discovery-enabled=false  # Disabled for permissioned network

# Static peers (other validators) - Provided by Riverbank
bootnodes=[
  "enode://VALIDATOR_1_PUBKEY@validator1.tramia.io:30303",
  "enode://VALIDATOR_2_PUBKEY@validator2.partner-bank.com:30303",
  "enode://VALIDATOR_3_PUBKEY@dr.tramia.io:30303",
  "enode://VALIDATOR_4_PUBKEY@regulatory-node.gov.ke:30303"
]

# ═══════════════════════════════════════════════════════════════════════
# VALIDATOR SETTINGS
# ═══════════════════════════════════════════════════════════════════════
miner-enabled=true
miner-coinbase="0xYOUR_VALIDATOR_ADDRESS"

# ═══════════════════════════════════════════════════════════════════════
# RPC API (Internal Only - Do NOT expose to internet)
# ═══════════════════════════════════════════════════════════════════════
rpc-http-enabled=true
rpc-http-host="127.0.0.1"
rpc-http-port=8545
rpc-http-api=["ETH","NET","QBFT","ADMIN","DEBUG"]
rpc-http-cors-origins=["*"]

rpc-ws-enabled=true
rpc-ws-host="127.0.0.1"
rpc-ws-port=8546
rpc-ws-api=["ETH","NET","QBFT"]

# ═══════════════════════════════════════════════════════════════════════
# METRICS (for monitoring)
# ═══════════════════════════════════════════════════════════════════════
metrics-enabled=true
metrics-host="0.0.0.0"
metrics-port=9545

# ═══════════════════════════════════════════════════════════════════════
# LOGGING
# ═══════════════════════════════════════════════════════════════════════
logging="INFO"
```

5. Key Management & Security

5.1 Validator Key Overview

Each validator has a unique cryptographic key pair that identifies it on the network and is used to sign blocks. This key is the most critical security asset - if compromised, an attacker could impersonate your validator.

> CRITICAL: The validator private key should NEVER leave the secure environment. Use HSM (Hardware Security Module) for production deployments.

5.2 Key Storage Options

Method	Security Level	Cost	Recommendation
File-based	Low (encrypted file)	Free	Development only
Cloud KMS	Medium-High	$100-300/mo	Staging/Small prod
Cloud HSM	High (FIPS 140-2 L3)	$500-1500/mo	Production
On-prem HSM	Highest	$10,000+ upfront	Enterprise/Regulated

5.3 Key Generation Ceremony

The initial key generation should follow a formal ceremony to ensure security and auditability:

```
KEY GENERATION CEREMONY PROCEDURE
═══════════════════════════════════════════════════════════════════════════

Participants Required:
  • Key Custodian (typically CTO or Security Officer)
  • Witness (second authorized person)
  • Optional: External auditor

Equipment Needed:
  • Air-gapped laptop (never connected to internet)
  • Hardware Security Module (HSM) or secure USB drive
  • Tamper-evident bags for backup storage

STEP 1: PREPARATION
──────────────────────────────────────────────────────────────────────────
☐ Verify air-gapped laptop has no network connectivity
☐ Boot from clean Ubuntu USB
☐ Install Besu tools from verified offline media

STEP 2: KEY GENERATION
```

```
────────────────────────────────────────────────────────────────────────────
# Generate new validator key pair
$ besu operator generate-blockchain-config \
    --config-file=qbft-config.json \
    --to=networkFiles \
    --private-key-file-name=validator-key

# Output:
#   networkFiles/keys/validator-key         (PRIVATE - protect this!)
#   networkFiles/keys/validator-key.pub     (PUBLIC - share with network)

STEP 3: KEY BACKUP
────────────────────────────────────────────────────────────────────────────
☐ Encrypt private key with strong passphrase
☐ Copy encrypted key to TWO separate USB drives
☐ Place USB drives in tamper-evident bags
☐ Store in separate physical locations (e.g., bank safe deposit boxes)

STEP 4: HSM IMPORT (Production)
────────────────────────────────────────────────────────────────────────────
☐ Import private key into HSM
☐ Verify HSM can sign test message
☐ Securely delete private key from air-gapped laptop
☐ Witness signs ceremony log

STEP 5: DOCUMENTATION
────────────────────────────────────────────────────────────────────────────
☐ Record public key fingerprint
☐ Document ceremony date, participants, backup locations
☐ Store ceremony log in secure document management system
```

5.4 Security Best Practices

- Network Isolation: Run validator on isolated network segment with strict firewall rules
- Minimal Attack Surface: Disable all unnecessary services and ports
- Regular Updates: Apply security patches within 7 days of release
- Access Control: Use SSH key authentication only, disable password login
- Audit Logging: Log all access and administrative actions
- Intrusion Detection: Deploy host-based IDS (e.g., OSSEC, Wazuh)
- Backup Testing: Quarterly test of key recovery from backup
- Key Rotation: Annual key rotation with governance approval

6. Deployment Options

6.1 Deployment Comparison

Factor	Cloud (AWS/Azure)	On-Premise	Hybrid
Setup Time	Hours to days	Weeks to months	Days to weeks
Monthly Cost	$800-1,500	$200-500 (power/net)	$500-1,000
Upfront Cost	None	$5,000-20,000	$2,000-10,000
Maintenance	Managed/low	Self-managed/higher	Mixed
Data Sovereignty	In-region possible	Full control	Flexible
Best For	Quick start, flexibility	Regulatory requirements	Balance of both

6.2 Cloud Deployment (Recommended for Starting)

```
CLOUD ARCHITECTURE (AWS Example)
═══════════════════════════════════════════════════════════════════════

┌──────────────────────────────────────────────────────────────────────┐
│                              AWS VPC                                 │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │                       Private Subnet                         │   │
│  │                                                              │   │
│  │   ┌──────────────────────────────────────────────────────┐   │   │
│  │   │                EKS Cluster / EC2 Instance            │   │   │
│  │   │   ┌──────────────────────────────────────────────┐   │   │   │
│  │   │   │              Besu Validator Container        │   │   │   │
│  │   │   │    • Hyperledger Besu 24.x                   │   │   │   │
│  │   │   │    • Connected to CloudHSM for signing       │   │   │   │
│  │   │   │    • Persistent EBS volume for chain data    │   │   │   │
│  │   │   └──────────────────────────────────────────────┘   │   │   │
│  │   └──────────────────────────────────────────────────────┘   │   │
│  │                                                              │   │
│  │   ┌──────────────────┐     ┌──────────────────┐              │   │
│  │   │   CloudHSM       │     │   CloudWatch     │              │   │
│  │   │   (Signing Keys) │     │   (Monitoring)   │              │   │
│  │   └──────────────────┘     └──────────────────┘              │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │                       Public Subnet                          │   │
│  │   ┌──────────────────┐                                       │   │
│  │   │   NAT Gateway    │  ← P2P traffic (port 30303)           │   │
│  │   │   + Security Grp │                                       │   │
│  │   └──────────────────┘                                       │   │
```

```
│   └──────────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────┘


Estimated Monthly Cost:
  • EC2 (m5.xlarge):     $140
  • EBS (500GB gp3):     $45
  • CloudHSM:            $1,000 (can share across services)
  • Data Transfer:       $50
  • CloudWatch:          $30
  ──────────────────────────────
  TOTAL:                 ~$1,265/month (with dedicated HSM)
                         ~$300/month (with KMS instead of HSM)
```

6.3 Docker Compose (Development/Testing)

```
# docker-compose.yml - Tramia Validator (Development)
version: '3.8'

services:
  validator:
    image: hyperledger/besu:24.1.0
    container_name: tramia-validator
    restart: unless-stopped
    command:
      - --config-file=/config/config.toml
      - --node-private-key-file=/secrets/validator.key
    ports:
      - "30303:30303/tcp"
      - "30303:30303/udp"
    volumes:
      - ./config:/config:ro
      - ./secrets:/secrets:ro
      - validator-data:/data/besu
    networks:
      - tramia-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8545/liveness"]
      interval: 30s
      timeout: 10s
      retries: 3

  prometheus:
    image: prom/prometheus:v2.45.0
    container_name: tramia-prometheus
    restart: unless-stopped
    ports:
      - "9090:9090"
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
```

```
      - prometheus-data:/prometheus
    networks:
      - tramia-network

  grafana:
    image: grafana/grafana:10.0.0
    container_name: tramia-grafana
    restart: unless-stopped
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
      - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards:ro
    networks:
      - tramia-network

volumes:
  validator-data:
  prometheus-data:
  grafana-data:

networks:
  tramia-network:
    driver: bridge
```

7. Operations & Maintenance

7.1 Operational Model

IMPORTANT: Validators run autonomously 24/7. They do NOT require dedicated operators or constant human supervision.

Task	Frequency	Human Involvement
Block production	Every 2 seconds	None - fully automated
Consensus voting	Every 2 seconds	None - fully automated
Health monitoring	Continuous	Automated alerts to on-call
Log review	Weekly	15-30 minutes
Security patches	Monthly	30-60 minutes
Software upgrades	Quarterly	1-2 hours (coordinated)
Key rotation	Annual	2-3 hours (ceremony)
Incident response	As needed (rare)	Remote troubleshooting

7.2 Staffing Requirements

Running a Tramia validator does NOT require dedicated staff. It can be managed by existing technical team members as part of their normal responsibilities:

```
STAFFING MODEL
═══════════════════════════════════════════════════════════════════════════


┌──────────────────────────────────────────────────────────────────────────┐
│                      EXISTING TEAM COVERAGE                              │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│   □□ DevOps / Platform Engineer                                          │
│      • Primary responsibility for validator infrastructure               │
│      • Part of normal on-call rotation                                   │
│      • Time commitment: 2-4 hours per month                              │
│                                                                          │
│   □□ Backend Engineers                                                   │
│      • Understand blockchain basics for integration work                 │
│      • Can assist with troubleshooting if needed                         │
│      • Time commitment: As needed (rare)                                 │
│                                                                          │
│   □□ Technical Lead / CTO                                                │
│      • Owns key ceremony decisions                                       │
│      • Approves governance votes                                         │
```

```
│      • Time commitment: 1-2 hours per quarter                          │
│                                                                         │
│   ✗ NO dedicated "Validator Operator" role required                    │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

7.3 Monitoring & Alerting

7.3.1 Key Metrics to Monitor

Metric	Alert Threshold	Severity
Node Up/Down	down > 2 minutes	CRITICAL - Page immediately
Peer Connections	< 2 peers	WARNING - Investigate
Missed Blocks (hourly)	> 5 blocks	WARNING - Check network
Block Height Lag	> 10 blocks behind	CRITICAL - Node syncing issue
Consensus Participation	< 95%	CRITICAL - Check validator
Disk Usage	> 80%	WARNING - Expand storage
Memory Usage	> 90%	WARNING - Check for leaks

7.3.2 Alert Configuration

```yaml
# prometheus/alerts.yml - Tramia Validator Alerts
groups:
  - name: tramia-validator
    rules:
      - alert: ValidatorOffline
        expr: up{job="tramia-validator"} == 0
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "Tramia validator is offline"
          description: "Validator has been down for more than 2 minutes"
          runbook: "https://docs.tramia.io/runbooks/validator-offline"

      - alert: LowPeerCount
        expr: besu_peers < 2
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Low peer connections"
          description: "Validator has only {{ $value }} peer connections"
```

```
    - alert: MissedBlocks
      expr: increase(besu_synchronizer_chain_head_number[1h]) < 1700
      labels:
        severity: warning
      annotations:
        summary: "Validator may be missing blocks"
        description: "Expected ~1800 blocks/hour, got {{ $value }}"

    - alert: DiskSpaceLow
      expr: (node_filesystem_avail_bytes / node_filesystem_size_bytes) * 100 < 20
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "Disk space low on validator"
        description: "Only {{ $value }}% disk space remaining"
```

8. Network Governance

8.1 Governance Model

Tramia uses a consortium governance model where major network changes require agreement from validator operators. This is implemented through on-chain voting.

8.2 Governance Actions

Action	Required Votes	Voting Period
Add new validator	3 of 4 (75%)	7 days
Remove validator	3 of 4 (75%)	7 days
Protocol upgrade	4 of 4 (100%)	14 days
Emergency action	3 of 4 (75%)	24 hours
Parameter change	3 of 4 (75%)	7 days

8.3 Voting Process

```
GOVERNANCE VOTING WORKFLOW
═══════════════════════════════════════════════════════════════════════

┌──────────────────────────────────────────────────────────────────────┐
│  STEP 1: PROPOSAL SUBMISSION                                         │
│  ────────────────────────────────────────────────────────────────────│
│  • Any validator can submit a proposal                               │
│  • Proposal includes: description, type, target, rationale           │
│  • Proposal registered on-chain with unique ID                       │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  STEP 2: DISCUSSION PERIOD (Optional)                                │
│  ────────────────────────────────────────────────────────────────────│
│  • Validators review proposal off-chain                              │
│  • Technical assessment conducted                                    │
│  • Questions/concerns addressed                                      │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│  STEP 3: VOTING PERIOD                                               │
│  ────────────────────────────────────────────────────────────────────│
│  • Each validator casts vote: FOR, AGAINST, or ABSTAIN              │
│  • Votes recorded on-chain                                           │
```

```
│  • Voting period: 7 days (standard) or 24 hours (emergency)              │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  STEP 4: EXECUTION                                                       │
│  ────────────────────────────────────────────────────────────────────────│
│  • If quorum reached and majority FOR: proposal passes                   │
│  • Action executed automatically by smart contract                       │
│  • All validators update their configuration                             │
└──────────────────────────────────────────────────────────────────────────┘
```

9. Troubleshooting Guide

9.1 Common Issues and Solutions

9.1.1 Node Won't Start

```
PROBLEM: Besu fails to start or crashes immediately

DIAGNOSIS:
$ docker logs tramia-validator --tail 100
$ journalctl -u besu -n 100

COMMON CAUSES & SOLUTIONS:

1. Invalid configuration file
   → Check config.toml syntax
   → Verify genesis.json matches other validators

2. Port already in use
   → netstat -tlnp | grep 30303
   → Stop conflicting service or change port

3. Insufficient memory
   → Increase Docker/VM memory allocation
   → Check for memory leaks in other services

4. Corrupted database
   → Stop node
   → Remove data directory
   → Restart (will resync from peers)
```

9.1.2 No Peer Connections

```
PROBLEM: Validator shows 0 peers

DIAGNOSIS:
$ curl -X POST http://localhost:8545 \
    -H "Content-Type: application/json" \
    -d '{"jsonrpc":"2.0","method":"net_peerCount","params":[],"id":1}'

COMMON CAUSES & SOLUTIONS:

1. Firewall blocking port 30303
   → Check: sudo ufw status
   → Fix: sudo ufw allow 30303/tcp && sudo ufw allow 30303/udp
```

```
2. Incorrect bootnode addresses
   → Verify enode URLs in config.toml
   → Confirm other validators are online

3. Network connectivity issues
   → Test: telnet validator2.tramia.io 30303
   → Check DNS resolution

4. Wrong network ID
   → Verify network-id matches genesis chainId
```

9.1.3 Not Producing Blocks

```
PROBLEM: Validator is connected but not participating in consensus

DIAGNOSIS:
$ curl -X POST http://localhost:8545 \
    -H "Content-Type: application/json" \
    -d
'{"jsonrpc":"2.0","method":"qbft_getValidatorsByBlockNumber","params":["latest"],"i
d":1}'

COMMON CAUSES & SOLUTIONS:

1. Validator not in active set
   → Check if your address is in validator list
   → May need governance vote to add

2. Key not loaded
   → Verify validator.key file exists and is readable
   → Check Besu logs for key loading errors

3. Clock skew
   → Ensure NTP is running: timedatectl status
   → Sync time: sudo ntpdate pool.ntp.org

4. Node still syncing
   → Check sync status: eth_syncing
   → Wait for sync to complete
```

9.2 Emergency Procedures

Scenario	Action
Node compromised	1. Immediately shut down node. 2. Notify Riverbank. 3. Initiate key rotation. 4. Investigate breach.

Key exposure	1. Stop validator. 2. Emergency governance vote to remove old key. 3. Generate new key via ceremony. 4. Add new validator.
Data corruption	1. Stop node. 2. Delete corrupted data. 3. Restart node (will resync from peers). 4. Monitor sync progress.
Network partition	1. Check network connectivity. 2. Verify firewall rules. 3. Contact other validators. 4. Network will resume when connectivity restored.

10. Getting Started Checklist

Follow this checklist to onboard as a Tramia validator:

```
TRAMIA VALIDATOR ONBOARDING CHECKLIST
═══════════════════════════════════════════════════════════════════════════

☐ PHASE 1: PREPARATION (Week 1)
──────────────────────────────────────────────────────────────────────────
  ☐ Review this documentation thoroughly
  ☐ Identify technical team members responsible for validator
  ☐ Choose deployment option (cloud vs on-premise)
  ☐ Provision infrastructure (server/VM + networking)
  ☐ Establish communication channel with Riverbank team

☐ PHASE 2: SETUP (Week 2)
──────────────────────────────────────────────────────────────────────────
  ☐ Install required software (Docker, Besu, monitoring tools)
  ☐ Receive genesis file and network configuration from Riverbank
  ☐ Conduct key generation ceremony
  ☐ Configure validator node
  ☐ Set up monitoring and alerting

☐ PHASE 3: TESTING (Week 3)
──────────────────────────────────────────────────────────────────────────
  ☐ Connect to Tramia testnet
  ☐ Verify block production and consensus participation
  ☐ Test failover and recovery procedures
  ☐ Validate monitoring alerts are working
  ☐ Complete security checklist

☐ PHASE 4: MAINNET ONBOARDING (Week 4)
──────────────────────────────────────────────────────────────────────────
  ☐ Submit validator public key to Riverbank
  ☐ Governance vote to add validator (7-day voting period)
  ☐ Upon approval, connect to mainnet
  ☐ Verify validator is producing blocks
  ☐ Handover to operations (BAU)

☐ ONGOING
──────────────────────────────────────────────────────────────────────────
  ☐ Participate in governance votes
  ☐ Apply security patches promptly
  ☐ Coordinate upgrades with network
  ☐ Annual key rotation
```

10.1 Contact Information

Purpose	Contact
Technical Onboarding	validators@tramia.io
Emergency Support	support@tramia.io (24/7)
Governance Coordination	governance@tramia.io
Documentation	https://docs.tramia.io/validators

Appendix A: Glossary

Term	Definition
Validator	A node that participates in block production and consensus
QBFT	Quorum Byzantine Fault Tolerant - the consensus algorithm used by Tramia
Consensus	The process by which validators agree on the next block
Block	A batch of transactions that are validated and added to the chain
Genesis	The first block of the blockchain, defines initial configuration
Finality	The point at which a transaction cannot be reversed
HSM	Hardware Security Module - secure device for storing cryptographic keys
Enode	Unique identifier for a node including its public key and network address
P2P	Peer-to-peer - direct communication between validator nodes
RPC	Remote Procedure Call - API for interacting with the node

END OF DOCUMENT

Riverbank Solutions Limited - Tramia Validator Technical Guide v1.0